The dressed normal moment integral with a continuous amplitude

Claude, with Daniel Murfet

2026-09-07

Abstract

Assembly of the continuous-amplitude milestone (Astra #16, route C; units 184–186). For $h, k\colon \mathrm{Fin}(m{+}1) \to \mathbb{N}$ with $k_i > 0$, ratios $\ell_i = (h_i + 1)/(2k_i)$, minimum λ attained on J, and a continuous amplitude η,

$$\frac{\int_{(0,1]^{m+1}} \eta(x)\, x^h e^{-\beta N x^{2k}}\, dx}{N^{-\lambda}(\log N)^{|J|-1}} \longrightarrow \frac{\Gamma(\lambda)\beta^{-\lambda}}{(|J|-1)!} \prod_{j\in J} \frac{1}{2k_j} \int_{(0,1]^{m+1}} \eta(P_J u) \prod_{i\notin J} u_i^{h_i - 2k_i\lambda}\, du$$

(`amplitude_tendsto`), where P_J zeroes the minimal coordinates. The limiting functional is *face-supported*: the amplitude is restricted to $u_J = 0$ and integrated over the nonminimal normal directions against the residual weight. This is the corrected leading coefficient behind Astra #16's qualification of the paper's eq. `thm_leading_coeff`; it reduces to $\eta(0)$ times the bare constant only when all ratios are equal. Zero `sorry`/`axiom`.

1 The things formalised

```
def faceProj (h k) (l) (u) : Fin d →   := fun i => if ratioExp h k i = l then 0 else u i
def residualWeight (h k) (l) (u) :   :=
   i, if ratioExp h k i = l then 1 else u i ^ ((h i :  ) - 2 * k i * l)
def faceLeadConst (h k) (l  ) :   :=
  Gamma l *   ^ (-l) / (multCount (ratioExp h k) l - 1).factorial
    *   i, if ratioExp h k i = l then 1 / (2 * k i) else 1
def amplitudeCoeff (h k) (l  ) ( ) :   :=
  faceLeadConst h k l   *   u in unitBox d,   (faceProj h k l u) * residualWeight h k l u
theorem face_moment_eq (h  :   i, ratioExp h k i = l →   i = 0) :
      u in unitBox d, (  i, faceProj h k l u i ^   i)
        * (faceLeadConst h k l   * residualWeight h k l u)
      = monomialMixedConst (fun i => h i +   i) k l
theorem face_moment_eq_zero (h  :   j, ratioExp h k j = l     j   0) : ... = 0
theorem amplitude_tendsto (d) (h k : Fin (d+1) →  ) (hk) (l  ) (hl) (h ) (hmin) (hatt) ( )
    (h  : Continuous  ) :
    Tendsto (fun N => (  x in unitBox (d + 1),   x * ((  i, x i ^ h i)
        * exp (-(  * N *   i, x i ^ (2 * k i)))))
      / (N ^ (-l) * log N ^ (multCount (ratioExp h k) l - 1))) atTop
      (  (amplitudeCoeff h k l   ))
```

2 Mathematics

The finite-N functionals are $T_N(\eta) = \int \eta(x)\, w_N(x)\, dx$ with the normalised weight $w_N = x^h e^{-\beta N x^{2k}}/(N^{-\lambda}(\log N)^{|J|-1})$; the limit is $T(\eta) = \int \eta(P_J u)\, c\, r(u)\, du$. Their monomial moments agree in the limit: for a face shift γ the moment of T is a product of one-dimensional integrals $\int_0^1 u_i^{h_i+\gamma_i-2k_i\lambda} du_i = 1/(h_i+\gamma_i+1-2k_i\lambda)$ over $i \notin J$ (and 1 over J), which is exactly the mixed constant of the shifted exponents (unit 184); for a non-face shift some factor is $0^{\gamma_j} = 0$ and the shifted moment tends to 0. The moment-transfer theorem of unit 185 (with eventual nonnegativity of w_N, i.e. $N > 1$) finishes.

3 Paper-facing meaning

This is the leading term of the paper's dressed normal moment at the level of one normal block, in full generality of exponent pattern: exponent λ, log degree $|J|-1$, and a face-supported residue/moment functional in place of origin evaluation. Combined with the bare results (units 175, 181, 183) the deterministic positive-orthant unit-cutoff programme now covers continuous amplitudes.

4 Lean notes

- `faceConst` already existed (2D cutoff file); the new constant is `faceLeadConst`. Grep the namespace before naming.
- `lake env lean` on a file uses the *stale* oleans of its imports: after changing a statement in an imported file, `lake build <Module>` that module first.
- `rw` cannot rewrite under `fun N =>` inside a `Tendsto`; prove the function equality by `funext` and rewrite with it.
- Goals from `Eventually.of_forall fun u => ?_` and from `integral_eq_zero_of_ae` arrive as beta-redexes (`(fun u => ...) u = 0 u`); `change` to the intended statement.
- `Integrable.fintype_prod_dep` and `integral_fintype_prod_eq_prod` on `Measure.pi` of the restricted Lebesgue measures (via `restrict_unitBox`) handle the product structure of the face functional without any coordinate splitting.